

Product Specialist Assessment

Ethereum Wallet Transaction Analysis

Candidate	Mario Youssef
Company	Koinly
Wallet	0xf23649b5DF0E12BA04c6fa1874B03265f79C636B
Blockchain	Ethereum Mainnet (Chain ID 1)

Table of Contents

- 1. API Selection & Data Sources
- 2. Transaction Export
- 3. Balance Calculation Methodology
- 4. Balance Comparison Results
- 5. Discrepancy Analysis
- 6. Discrepancy Resolution (Bonus)
- 7. Technical Implementation

1. API Selection & Data Sources

To pull the complete transaction history for the wallet, I used the **Etherscan API V2** as the primary data source. Etherscan is the most widely used Ethereum block explorer and provides a free-tier API with comprehensive coverage of all on-chain activity.

Etherscan Endpoints Used:

Endpoint	Purpose	API Action
Normal Transactions	ETH transfers, contract calls, gas fees	txlist
Internal Transactions	ETH moved by smart contracts (e.g. DEX swaps)	txlistinternal
ERC20 Token Transfers	All token movements (USDC, DAI, etc.)	tokenx

Additionally, I integrated an **Alchemy archive node** to query historical on-chain balances at specific block numbers using the `balanceOf(address)` function via `eth_call`. This enables verifying the exact token balance before and after each transaction, which is critical for understanding tokens with non-standard transfer mechanics (tax tokens, rebasing tokens).

Rate Limiting & Pagination:

The free Etherscan API tier allows ~5 requests per second. The script handles this with a 0.22-second delay between calls and automatic retry on rate-limit errors. Pagination is handled by fetching up to 10,000 records per page and continuing until fewer records are returned.

2. Transaction Export

All transactions are exported to both **CSV** and **styled Excel (.xlsx)** formats. NFT transactions (ERC721/ERC1155) are automatically detected and skipped by checking for the presence of a `tokenId` field without a `tokenDecimal` field.

Transaction Summary:

Metric	Value
Total Transactions Exported	606
Normal ETH Transactions	255
Internal Transactions	78
ERC20 Token Transfers	273
Date Range	Sept 2021 - Jan 2025
Unique Tokens	30+

CSV Columns:

Column	Description
Date	UTC timestamp of the transaction
Block Number	Ethereum block containing the transaction

Column	Description
Transaction Hash	Unique transaction identifier
Type	IN, OUT, IN (internal), OUT (internal), OUT (failed)
From / To	Sender and recipient addresses
Amount	Token or ETH amount transferred
Token/Asset	Token symbol (e.g. ETH, USDC, SOHM)
Gas Fee (in ETH)	Gas cost for the sender
Function	Smart contract function called
Input Data	Raw calldata of the transaction

The Excel output includes color-coded rows: **green for incoming**, **red for outgoing**, and gray for failed transactions. Headers are styled with blue backgrounds and the sheet includes frozen panes and auto-filter for easy navigation.

3. Balance Calculation Methodology

ETH Balance:

The expected ETH balance is calculated by summing all inflows and subtracting all outflows and gas fees:

```
Expected ETH = SUM(ETH IN) + SUM(ETH IN internal) - SUM(ETH OUT) - SUM(ETH OUT internal) - SUM(Gas Fees)
```

ERC20 Token Balances:

For each ERC20 token, the expected balance is calculated from Transfer events:

```
Expected Token Balance = SUM(IN transfers) - SUM(OUT transfers)
```

The **actual on-chain balance** is fetched via the Etherscan API for ETH and via Alchemy's archive node (`balanceOf()`) for ERC20 tokens. Any difference between expected and actual indicates a discrepancy that requires investigation.

Historical Balance Verification:

For the discrepancy audit trail, the script queries the Alchemy archive node at both **block N-1** (before the transaction) and **block N** (after the transaction) for each token transfer. This provides a complete picture of how the on-chain balance changed at each step, which is essential for understanding non-standard token mechanics.

4. Balance Comparison Results

The following table shows all assets where the expected balance (from Transfer events) differs from the actual on-chain balance. Tokens that match are omitted for brevity.

Token	Expected Balance	Actual Balance	Difference	Status
ETH	0.0209...	0.0209...	~0	OK
sOHM	-1.9896	0	+1.9896	DISCREPANCY
WAGMIGAMES	32,395.14	2,921.29	-29,473.85	DISCREPANCY
DETF	1,152.52	1,207.91	+55.39	DISCREPANCY
Berry	5,000	0	-5,000	DISCREPANCY
DEgg	2,000	0	-2,000	DISCREPANCY
KiCT	1,250	0	-1,250	DISCREPANCY
TOSH	1,800.39	0	-1,800.39	DISCREPANCY

7 tokens show discrepancies between expected and actual balances. The next section analyzes each one and explains the root cause.

5. Discrepancy Analysis

Each discrepancy falls into one of four categories based on the relationship between expected and actual balances. The script automatically classifies each one:

Category	Pattern	Count
Rebasing / Staking Rewards	Expected < 0, Actual >= 0	1 (sOHM)
Scam / Airdrop Token (Burned)	Expected > 0, Actual = 0	4 (Berry, DEgg, KICT, TOSH)
Non-Transfer Gains	Expected > 0, Actual > Expected	1 (DETF)
Fee-on-Transfer Tax Token	Expected > 0, Actual < Expected	1 (WAGMIGAMES)

5.1 sOHM (Staked OHM) - Rebasing Token

sOHM is the staked version of Olympus DAO's OHM token. It uses a **rebasing mechanism** where the token balance automatically increases every ~8 hours (epoch) to distribute staking rewards. These increases happen by adjusting a global index, not by Transfer events.

Event	Date	Amount	On-Chain Balance
Stake (IN)	2021-10-16	+2.029 sOHM	(N-1) 0 / (N) 2.029
Rebase rewards	Oct 2021 - Dec 2023	No Transfer events	Balance grew to ~4.019
Swap (OUT)	2023-12-17	-4.019 sOHM	(N-1) 4.019 / (N) 0

Root cause: The wallet staked 2.029 sOHM and over ~2 years the balance nearly doubled to 4.019 sOHM via rebasing. Since rebases don't emit Transfer events, the script only sees 2.029 IN and 4.019 OUT, resulting in an expected balance of -1.989. The actual balance is 0 (all swapped out). The +1.989 gap is exactly the rebase rewards.

5.2 WAGMIGAMES - Fee-on-Transfer Tax Token

WAGMIGAMES is a reflection token built on the Reflect Finance (RFI) model. Its smart contract applies a **configurable tax** on buys and sells via the `updateFees()` function. The contract owner changed the fees 7 times (verified on-chain). At the time of the sell, the fee was 2% (LP only).

Transaction	Date	Configured Fee	Effective Fee	Notes
Buy 1	2023-12-22	25%	0%	Routed via aggregator (fee-excluded path)
Buy 2	2024-01-13	25%	0%	Routed via aggregator (fee-excluded path)
Sell	2025-01-16	2%	2%	0% RFI, 0% marketing, 0% dev, 0% team, 2% LP

The on-chain balances confirm the buys were not taxed: the `Transfer` event amounts match the `balanceOf()` changes exactly. The buys were routed through DEX aggregator contracts that were excluded from fees. Only the sell was subject to the 2% LP fee active at that time.

TX	Date	Transfer Event Amount	On-Chain (N-1)	On-Chain (N)
Buy 1 (IN)	2023-12-22	+9,583,915	0	9,583,915
Buy 2 (IN)	2024-01-13	+6,792,819	9,584,274	16,377,093
Sell (OUT)	2025-01-16	-16,344,339	16,380,014	2,921

Tax calculation from the sell transaction (2% fee at the time):

```
On-Chain(N-1) 16,380,014 - Transfer 16,344,339 - On-Chain(N) 2,921 = 32,754 WAGMIGAMES taken by contract
```

Root cause: The Transfer events log the pre-tax amount, but the smart contract's `_transferTokens()` function deducts fees internally. The `balanceOf()` is not a stored value but a computed result based on a reflection rate that changes with every taxed transfer. The nominal 2% fee on 16.3M tokens should be ~327K, but only 32,754 (~0.2%) was deducted. This is because the fee is applied through the reflection math, not as a direct percentage of the balance. The `swapBack()` that triggers within the same transaction shifts the reflection rate, causing the tax to manifest non-linearly.

5.3 DETF - Reflection/Redistribution Gains

DETF uses a redistribution mechanism similar to WAGMIGAMES. The wallet received 1,152.52 DETF via a single Transfer event, but the actual on-chain balance grew to 1,207.91 DETF over time. The +55.39 extra tokens came from passive reflection rewards (redistribution from other holders' transactions), which do not emit Transfer events.

5.4 Berry, DEgg, KiCT, TOSH - Scam/Airdrop Tokens

These four tokens share the same pattern: the wallet received them via a single unsolicited Transfer event (airdrop), but the actual on-chain balance is now **zero**. No outgoing Transfer event exists for any of them.

Token	Received	Date	Expected	Actual
Berry	5,000	2022-04-03	5,000	0
DEgg	2,000	2022-03-22	2,000	0
KiCT	1,250	2022-03-25	1,250	0
TOSH	1,800.39	2022-03-30	1,800.39	0

Root cause: These are scam/airdrop tokens. The token contracts include admin functions that can burn or zero-out holder balances without emitting Transfer events. They are worthless and were never interacted with by the wallet owner.

6. Discrepancy Resolution (Bonus)

All 7 discrepancies are **automatically resolved** by the script using pattern-matching logic that classifies each discrepancy based on the relationship between expected and actual balances. No hardcoded token-specific rules are needed.

Auto-Detection Logic:

Condition	Classification	Resolution
Expected < 0 Actual >= 0	Rebasing / Staking	Balance increased via non-transfer mechanism (rebase epochs, staking yield). Gap equals rewards earned.
Expected > 0 Actual = 0	Scam / Admin Burn	Token balance zeroed outside of Transfer events. Likely scam airdrop with admin burn capability.
Expected > 0 Actual > Expected	Reflection Gains	Non-transfer balance increase from passive redistribution rewards or direct contract minting.
Expected > 0 Actual < Expected	Fee-on-Transfer Tax	Transfer events log pre-tax amounts but wallet receives less due to built-in smart contract tax.

Resolution Summary:

Token	Discrepancy	Resolution
sOHM	+1.989 sOHM	Rebasing rewards over 2 years (staking yield)
WAGMIGAMES	-32,754.18 WAGMI	Fee-on-transfer tax (2% on sell, buys untaxed)
DETF	+55.39 DETF	Passive reflection redistribution rewards
Berry	-5,000	Scam airdrop token, admin burn
DEgg	-2,000	Scam airdrop token, admin burn
KiCT	-1,250	Scam airdrop token, admin burn
TOSH	-1,800.39	Scam airdrop token, admin burn

On-Chain Verification:

To verify these resolutions, the script queries the Alchemy archive node for historical `balanceOf()` at each transaction's block. For every token transfer, the report shows the balance at **block N-1** (before) and **block N** (after), enabling direct verification of how much the smart contract actually credited or debited. This is especially valuable for tax tokens like WAGMIGAMES, where the calculation $\text{On-Chain}(N-1) - \text{Transfer Amount} - \text{On-Chain}(N) = \text{Tax Taken}$ precisely quantifies the contract's fee.

7. Technical Implementation

Technology Stack:

Component	Technology
Language	Python 3
APIs	Etherscan API V2 + Alchemy Archive Node (JSON-RPC)

Component	Technology
Excel Styling	openpyxl
Precision	Python Decimal (36 digits) for accurate token amounts
Dependencies	Only openpyxl (all else is Python standard library)

Output Files:

```
transactions/  
  transactions.csv  
  transactions.xlsx  
discrepancies/  
  discrepancies.csv  
  discrepancies.xlsx
```

How to Run:

```
1. pip install openpyxl  
2. Create .env with ETHERSCAN_API_KEY, WALLET_ADDRESS, ALCHEMY_API_KEY  
3. python script.py
```